DATA 442/610: Deep Learning and Neural Networks
Assignment 4: VAEs

Assigned: Nov 9, 2026; Due: Nov 25, 2026

Instructions

This assignment will be graded out of 100 points. It contains one problems. Your score will be calculated based on the parts you turn in, up to a maximum of 100 points. **Any bonus questions will be scored and added on top of the 100-point grade total.**

Please submit a Jupyter notebook with labeled headings/subheadings for each part. This assignment will be graded based on demonstrated understanding of the core concepts. You do not need to achieve the best possible neural network accuracy. High marks will come from thoughtful documentation and explanation of *what* you are attempting, *how* you expect it to perform, and *why* you believe it will behave like it does. **Please comment your code**.

This assignment may require working out some intermediate mathematical steps. You are encouraged to show your work on any derivations underlying your code, and include this alongside your submission. You may submit any mathematical derivations as typeset equations in your Jupyter Notebooks. If you prefer not to type out that many equations, you may include a scanned PDF of handwritten calculations.

Note for Problem Set 4: The ML models in this assignment can reach reasonable accuracy within 50-100 epochs (∼15 minutes on the W&M HPC Hima partition using the SLURM skeleton below), although you may train longer if you like. **If you use SLURM or run an external script, please submit your Python and/or sbatch files alongside your Jupyter notebook**.

```
#!/bin/bash
#SBATCH --job-name=ps4_vae
#SBATCH --nodes=1
#SBATCH --ntasks=8
#SBATCH --gres=gpu:1
#SBATCH --mem=16G
#SBATCH --time=4:00:00

source ~/.bashrc
module load cuda/12.3
conda activate ~/envs/nndl_env # Activate your environment

python train_vae.py << your arguments here >>
```

Problem 1 (100 points) - Variations on Variational Autoencoders

Part 1: Data preprocessing (10 points)

The dsprites dataset is available at `https://github.com/google-deepmind/dsprites-dataset/raw/refs/heads/master/dsprites_ndarray_co1sh3sc6or40x32y32_64x64.npz`. Load the dataset into your environment and separate it into training, validation, and testing datasets. Make sure to print out the size of each data subset. Select a few random examples from your training dataset. Visualize them and comment on the variation you observe.

Part 2: Baseline VAE (30 points)

Implement a variational autoencoder with a latent bottleneck of size 10 and train it on the dsprites dataset. You may use the example from class as a guide. You are welcome to use any architecture: CNN, MLP, Vision Transformer, etc. Explain why you chose your particular encoder and decoder model and why you expect it to work well on this task. Train the model to minimize the VAE loss (maximize the evidence lower bound).

$$\mathcal{L}_{\text{VAE}} = -\mathbb{E}\big[\log q_\phi(x|z)\big] + D_{KL}\big[p_\theta(z|x) \,\|\, q(z)\big]$$

For the data log-likelihood, you may use a Bernoulli, Gaussian, or Laplace distribution. Make sure to note which one you chose and how it affects the log-likelihood calculation. Likewise, comment on any additional training hyperparameters you used.

While training, track the total loss, as well as its two subcomponents: the data log-likelihood (or reconstruction term) as well as the KL divergence between the latent variable distribution and its prior (the regularization term). You may find it helpful to write a separate script to perform training on the HPC cluster, rather than within your notebook.

Once finished, plot training and validation curves for each metric: loss, data log-likelihood, and KL divergence. In addition, select a batch of samples from your test dataset and plot the original sample alongside the reconstruction returned by your model. How well does your model do?

Part 3: β-VAE (10 points)

A β-VAE adjusts the weight on the KL divergence term to control the strength of regularization. That is, the model minimizes the loss

$$\mathcal{L}_{\beta\text{-VAE}} = -\mathbb{E}\big[\log q_\phi(x|z)\big] + \beta\, D_{KL}\big[p_\theta(z|x) \,\|\, q(z)\big]$$

Modify your VAE into a β-VAE. Repeat your training for at least two values of $\beta \neq 1$, and plot the training curves and reconstructed test examples. Comment on how this modification affects the training and inference behavior of your models.

Part 4: Analysis (20 points)

For your VAE and β-VAE models, generate the following plots:

Latent traversal: Select an example from your test dataset and use your encoder to predict a vector of latent features. Conduct a traversal over each latent feature, by systematically vary its value over some range while holding all other feature values constant. Decode each traversal into a set of generated images and plot them next to each other. The final plot should have 10 rows (one for each latent feature) and N columns, where N is the size of your traversal. Comment on the behavior induced by varying individual features.

Capacity plot: The regularization term in the VAE loss measures the capacity C of the latent features to transmit information about the inputs. For each latent feature z_i, we can similarly measure $C_i = D_{KL}\big[p_\theta(z_i|x)\|q(z_i)\big]$ (if $C_i = 0$ the feature is indistinguishable from its Gaussian prior and transmits no information about the input). Using your test dataset, estimate and plot the capacity C_i for each latent feature. How many features are "informative" about the input?

Correlation plot: The dsprites dataset was generated using 6 latent *factors*: [color, shape, scale, orientation, position (x), and position (y)]. These are accessible via the field `latents_values` in the dsprites NPZ file. Using your test dataset, measure and plot the correlation between VAE latent variables and the ground truth latent factors (you may omit color as it is white for all examples). The plot should be a heatmap with 10 rows (one for each latent feature) and 5 columns (one for each ground truth factor). How well do your models capture and encode this generative information?

Part 5: Disentangling VAEs (30 points)

Note on scoring: Here you may choose from three possible options. If you implement more than one, you will receive up to an additional 10 bonus points per extra method (so a maximum of 20 possible bonus points in total).

A *disentangled* representation has latent features that are uniquely sensitive to individual data-generating factors. Several approaches have been proposed to improve disentangled representation learning. In this part, you will implement any of the following three methods (your choice). You should train the model, plot its loss curves and sample test reconstructions, and generate the three plots from Part 4: latent traversal capacity plot, and correlation plot. Compare the performance of the disentangling VAEs to the baseline VAE and β-VAE.

Option 1 - Factor-VAE: In the Factor-VAE paper, the authors augment the objective to penalize latent distributions that do not factorize into a product of marginals $p_\theta(z) = \prod_j p_\theta(z_j)$. The belief is that if features from disentangled representations represent independent generative factors, then the overall distribution of representations should factorize into a product of feature distributions. The model aims to minimize the augmented objective

$$\mathcal{L}_{\text{Factor-VAE}} = -\mathbb{E}\big[\log q_\phi(x|z)\big] + D_{KL}\big[p_\theta(z|x)\,\|\,q(z)\big] + \gamma\, D_{KL}\big[p_\theta(z)\,\|\,\prod_j p_\theta(z_j)\big]$$

The last term penalizes **total correlation** among latent variables with a hyperparameter γ. Calculating this requires estimating the marginal $p_\theta(z) = \sum_x p_\theta(z|x)p(x)$ which depends on the full dataset. To do this, Factor VAE samples a batch from $p_\theta(z)$ via the encoder, and then permutes each latent dimension of the batch to approximate the dimension-wise distributions $p_\theta(z_j)$ (see below).

```
def permute_latent_dims(z):
    B, N = z.shape
    z_perm = []
    for ii in range(N):
        perm = torch.randperm(B).to(z.device)
```

```
        z_perm_ii = z[perm, ii]
        z_perm.append(z_perm_ii)
    return torch.stack(z_perm, dim=1)
```

To distinguish between z and z_{perm}, Factor-VAE trains a discriminator model $D_\psi(z)$ to minimize

$$\mathcal{L}_D = \frac{1}{B}\sum_{i=1}^{B} \log\left[D_\psi(z^{(i)})\right] + \frac{1}{B}\sum_{j=1}^{B} \log\left[1 - D_\psi(z^{(j)}_{\text{perm}})\right]$$

Similar to GAN, this discriminator is trained alongside the VAE model. Using the trained discriminator, one can calculate the total correlation term in $\mathcal{L}_{\text{Factor-VAE}}$ as

$$D_{KL}\left[p_\theta(z) \,\|\, \prod_j p_\theta(z_j)\right] \approx \frac{1}{B}\sum_{i=1}^{B} \log\left[D(z^{(i)})\right] - \log\left[1 - D(z^{(i)})\right]$$

Option 2 - VAE with controlled capacity increase (CC-VAE): In the CC-VAE paper, the authors propose explicitly constraining the latent capacity of the network to a target $C(t)$ that increases slowly over time. The belief is that this promotes disentangled representations because the model will only "open up" a latent feature dimension when it is allowed the capacity to do so. At each step the model aims to minimize

$$\mathcal{L}_{\text{CC-VAE}} = -\mathbb{E}\left[\log q_\phi(x|z)\right] + \gamma \left| D_{KL}\left[p_\theta(z|x) \,\|\, q(z)\right] - C(t) \right|$$

Here γ is taken to be some large constant to enforce a strict constraint (e.g. $\gamma = 1000$). The capacity $C(t)$ typically follows linear annealing up to some maximum value, for example:

$$C(t) = \min\left(C_{\min} + \frac{C_{\max} - C_{\min}}{\text{max steps}} \times t,\ C_{\max}\right)$$

Option 3 - Total Correlation VAE (TC-VAE): In the TC-VAE paper, the authors decompose the KL divergence term in the VAE loss into three components.

$$D_{KL}\left[p_\theta(z|x) \,\|\, q(z)\right] = \underbrace{D_{\text{KL}}\left[p_\theta(z,x) \,\|\, p_\theta(z)p(x)\right]}_{\text{data-latent MI}} + \underbrace{D_{\text{KL}}\left[p_\theta(z) \,\|\, \prod_j p_\theta(z_j)\right]}_{\text{total correlation}} + \underbrace{\sum_j D_{\text{KL}}\left[p_\theta(z_j) \,\|\, p(z_j)\right]}_{\text{dimension-wise KL}}$$

The first term is the mutual information between the data x and the latent representation z. The second measures total correlation as the KL divergence between the overall latent distribution $p_\theta(z)$ and the product of its factors. The third sums the KL divergence along each feature dimension by aggregating the difference between $p_\theta(z)$ and its prior. The belief is that applying a large weight β to only the second term in this decomposition promotes decorrelated and disentangled representations.

This equation requires estimating the marginal distribution $p_\theta(z) = \sum_x p_\theta(z|x)p(x)$, which depends on the full dataset (and likewise for $p_\theta(z_j)$). The TC-VAE authors propose a minibatch estimator that you may use (here B is batch size and N is dataset size)

$$\mathbb{E}\left[\log p_\theta(z)\right] \approx \frac{1}{B}\sum_{i=1}^{B}\left[\log \frac{1}{NB}\sum_{j=1}^{M} p_\theta\left(z(x^{(i)}) \,|\, x^{(j)}\right)\right]$$

For latent variable batch $z^{(i)}$, $i = 1, \ldots, B$ sampled from means $\mu(i)$ and variances $\sigma^{2,(i)}$, this requires computing $\mathcal{N}(z^i \,|\, \mu^{(j)}, \sigma^{2,(j)})$ for each i, j pair. This is efficiently done using tensor broadcasting.